

响应报文时间

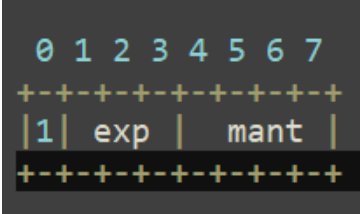
题目描述

IGMP 协议中，有一个字段称作最大响应时间 (Max Response Time)，HOST收到查询报文，解析出 MaxResponsetime 字段后，需要在 (0, MaxResponseTime] 时间 (s) 内选取随机时间回应一个响应报文，如果在随机时间内收到一个新的查询报文，则会根据两者时间的大小，选取小的一方刷新回应时间。

最大响应时间有如下计算方式：

当 Max Resp Code < 128, Max Resp Time = Max Resp Code ;

当 Max Resp Code ≥ 128,



Max Resp Time = (mant | 0x10) << (exp + 3);

注: exp最大响应时间的高5~7位: mant 为最大响应时间的低4位。

其中接收到的MaxRespCode 最大值为 255，以上出现所有字段均为无符号数。

现在我们认为 HOST收到查询报文时，选取的随机时间必定为最大值，现给出 HOST 收到查询报文个数 C，HOST 收到该报文的时间T，以及查询报文的最大响应时间字段值 M，请计算出HOST 发送响应报文的时间。

输入描述

第一行为查询报文个数 C，后续每行分别为 HOST 收到报文时间 T，及最大响应时间M，以空格分割。

输出描述

HOST 发送响应报文的时间。

搜索 AI提问 评论 笔记

▼

备注

用例确定只会发送一个响应报文， 不存在计时结束后依然收到查询报文的情况。

用例

输入	3 0 20 1 10 8 20
输出	11
说明	收到3个报文， 第0秒收到第1个报文，响应时间为20秒，则要到0+20=20秒响应； 第1秒收到第2个报文，响应时间为10秒，则要到1+10=11秒响应，与上面的报文的响应时间比较获得响应时间最小为11秒； 第8秒收到第3个报文，响应时间为20秒，则要到8+20=28秒响应，与第上面的报文的响应时间比较获得响应时间最小为11秒； 最终得到最小响应报文时间为11秒

输入	2 0 255 200 60
输出	260
说明	收到2个报文， 第0秒收到第1个报文，响应时间为255秒，则要到(15 0x10) << (7 + 3)= 31744秒响应; (mant = 15, exp =7) 第200秒收到第2个报文，响应时间为60秒，则要到200+60-260秒响应，与第上面的报文的响应时间比较获得响应时间最小为260秒： 最终得到最小响应报文时间为260秒

题目解析

本题主要考察字符串操作。

每一个报文都有一个最迟响应时间，即 $t + \text{getMaxResponseTime}(m)$ ，我们只要求出最小的最迟响应时间即可。

关于 $\text{getMaxResponseTime}$ 的逻辑：

当 $m < 128$ 时， m 就是 MaxResponseTime

当 $m \geq 128$ 时，需要将 m 转化为八位二进制字符串，取 1~3 范围作为 exp ，取 4~7 范围作为 mant

然后再将 exp 和 mant 从二进制串转化为十进制，带入公式： $(\text{mant} | 16) \ll (\text{exp} + 3)$ 求解即可。

Java 算法源码

```
1 import java.util.Scanner;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         int c = sc.nextInt();
8
9         int[][] messages = new int[c][2];
10        for (int i = 0; i < c; i++) {
11            messages[i][0] = sc.nextInt();
12            messages[i][1] = sc.nextInt();
13        }
14
15        System.out.println(getResult(messages));
16    }
17
18    public static int getResult(int[][] messages) {
19        int ans = Integer.MAX_VALUE;
20    }
```

```

21     for (int[] message : messages) {22         int respTime = message[0] + getMaxResponseTime(message[1]);
23         ans = Math.min(ans, respTime);
24     }
25
26     return ans;
27 }
28
29 public static int getMaxResponseTime(int m) {
30     if (m >= 128) {
31         StringBuilder bStr = new StringBuilder(Integer.toBinaryString(m));
32
33         int exp = Integer.parseInt(bStr.substring(1, 4), 2);
34         int mant = Integer.parseInt(bStr.substring(4), 2);
35
36         return (mant | 16) << (exp + 3);
37     } else {
38         return m;
39     }
40 }
41 }

```

JS算法源码

```

1  /* JavaScript Node ACM模式 控制台输入获取 */
2  const readline = require("readline");
3
4  const rl = readline.createInterface({
5      input: process.stdin,
6      output: process.stdout,
7  });
8
9  const lines = [];
10 let c;

```

运行

```
11 | rl.on("line", (line) => {12 |   lines.push(line);
13 |
14 |   if (lines.length == 1) {
15 |     c = parseInt(lines[0]);
16 |   }
17 |
18 |   if (c && lines.length == c + 1) {
19 |     lines.shift();
20 |     const messages = lines.map((s) => s.split(" ").map(Number));
21 |
22 |     console.log(getResult(messages));
23 |
24 |     lines.length = 0;
25 |   }
26 | });
27 |
28 | function getResult(messages) {
29 |   let ans = Infinity;
30 |
31 |   for (let [t, m] of messages) {
32 |     const respTime = t + getMaxResponseTime(m);
33 |     ans = Math.min(ans, respTime);
34 |   }
35 |
36 |   return ans;
37 | }
38 |
39 | function getMaxResponseTime(m) {
40 |   if (m >= 128) {
41 |     let bStr = Number(m).toString(2);
42 |
43 |     const exp = parseInt(bStr.slice(1, 4), 2);
44 |     const mant = parseInt(bStr.slice(4), 2);
45 |
46 |     return (mant | 16) << (exp + 3);
47 |   } else {
48 |     return m;
```

```
49 | }  
    | 50 | }
```

Python算法源码

```
1  import sys  
2  
3  # 输入获取  
4  c = int(input())  
5  messages = [list(map(int, input().split())) for _ in range(c)]  
6  
7  
8  def getMaxResponseTime(m):  
9      if m >= 128:  
10         bStr = bin(m)[2:]  
11  
12         exp = int(bStr[1:4], 2)  
13         mant = int(bStr[4:], 2)  
14  
15         return (mant | 16) << (exp + 3)  
16     else:  
17         return m  
18  
19  
20 # 算法入口  
21 def getResult():  
22     ans = sys.maxsize  
23  
24     for t, m in messages:  
25         respTime = t + getMaxResponseTime(m)  
26         ans = min(ans, respTime)  
27  
28     return ans
```

```
29 | 30 |
31 | # 算法调用
32 | print(getResult())
```

C算法源码

```
1  #include <stdio.h>
2  #include <limits.h>
3  #include <math.h>
4
5  int getMaxResponseTime(int m) {
6      // Max Resp Code >= 128时, 最大响应时间 = (mant | 0x10) << (exp + 3);
7      if (m >= 128) {
8          // 将十进制数 m 转成 二进制字符串对应的整型数组, 比如 m = 255时, bin[8] = {1, 1, 1, 1, 1, 1, 1, 1};
9          int bin[8] = {0};
10
11         int i = 7;
12         while (m > 0) {
13             bin[i--] = m % 2;
14             m /= 2;
15         }
16
17         //      j      0  1  2  3  4  5  6  7
18         // bin[] = {1, 1, 1, 1, 1, 1, 1, 1};
19         // exp截取bin的 j:1~3 范围, 注意对应范围当成一个新的二进制数, 因此j=3的位置是2^0
20         int exp = 0;
21         for (int j = 1; j < 4; j++) {
22             exp += bin[j] * (1 << (3 - j));
23         }
24
25         // mant截取bin的 j: 4~7 范围, , 注意对应范围当成一个新的二进制数, 因此j=7的位置是2^0
26         int mant = 0;
27         for (int j = 4; j < 8; j++) {
```

运行

```

28         mant += bin[j] * (1 << (7 - j));
29     }
30
31     // 0x10 等价于 16
32     return (mant | 16) << (exp + 3);
33 } else {
34     return m;
35 }
36 }
37
38 int main() {
39     int c;
40     scanf("%d", &c);
41
42     int res = INT_MAX;
43
44     int t, m;
45     for (int i = 0; i < c; i++) {
46         scanf("%d %d", &t, &m);
47         int respTime = t + getMaxResponseTime(m);
48         res = (int) fmin(res, respTime);
49     }
50
51     printf("%d\n", res);
52
53     return 0;
54 }
55

```

C++ 算法源码

```

1 #include <bits/stdc++.h>
2
3 using namespace std;

```



```

4
5 | int getMaxResponseTime(int m) {
6 |     // Max Resp Code >= 128时, 最大响应时间 = (mant | 0x10) << (exp + 3);
7 |     if (m >= 128) {
8 |         // 将十进制数 m 转成 二进制字符串对应的整型数组, 比如 m = 255时, bin[8] = {1, 1, 1, 1, 1, 1, 1, 1};
9 |         vector<int> bin(8, 0);
10
11 |         int i = 7;
12 |         while (m > 0) {
13 |             bin[i--] = m % 2;
14 |             m /= 2;
15 |         }
16
17 |         //      j      0  1  2  3  4  5  6  7
18 |         // bin[] = {1, 1, 1, 1, 1, 1, 1, 1};
19 |         // exp截取bin的 j:1~3 范围, 注意对应范围当成一个新的二进制数, 因此j=3的位置是2^0
20 |         int exp = 0;
21 |         for (int j = 1; j < 4; j++) {
22 |             exp += bin[j] * (1 << (3 - j));
23 |         }
24
25 |         // mant截取bin的 j: 4~7 范围, , 注意对应范围当成一个新的二进制数, 因此j=7的位置是2^0
26 |         int mant = 0;
27 |         for (int j = 4; j < 8; j++) {
28 |             mant += bin[j] * (1 << (7 - j));
29 |         }
30
31 |         // 0x10 等价于 16
32 |         return (mant | 16) << (exp + 3);
33 |     } else {
34 |         return m;
35 |     }
36 | }
37
38 | int main() {
39 |     int c;
40 |     cin >> c;
41

```

```
42     int res = INT_MAX;
43
44     int t, m;
45     for (int i = 0; i < c; i++) {
46         cin >> t >> m;
47         int respTime = t + getMaxResponseTime(m);
48         res = min(res, respTime);
49     }
50
51     cout << res << endl;
52
53     return 0;
54 }
```